

TP N° 4

Dataframes et fichiers CSV

But du TP :

- Manipuler des dataframes.
- Importer / exporter des données depuis / vers des fichiers csv.

Énoncé :

A. Dataframes

Les dataframes sont, comme les matrices, des structures de données constituées de lignes et colonnes. Elles sont utilisées pour stocker des tables de données telles que celles utilisées par les tableaux. Par la suite, nous les utiliserons pour stocker les info des datasets pour entraîner et tester les modèles ML.

Exemple : on veut créer un dataframe pour stocker les informations de la table suivante :

Type de marchandise	disponibilité &tonne'	volume &m³	profit &par tonne'	origine
semoule	18	400	2000	France
riz	10	300	2500	Inde
farine	5	200	5000	France
lentille	20	500	3500	Bresil

Pour créer un dataframe on utilise la fonction **data.frame()** qui prend comme arguments :

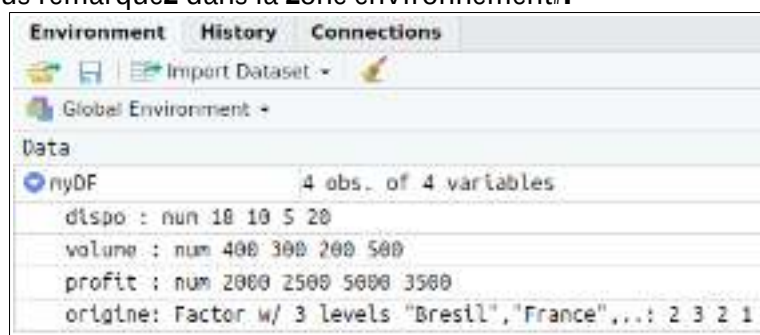
- les vecteurs définissant les colonnes avec possibilité de leur donner des noms.
- l'argument **row.names** qui permet de spécifier les noms des lignes. Cet argument est optionnel mais très recommandé pour simplifier la manipulation par ligne.

Le code qui suit permet de le faire et on a choisi d'utiliser le type de marchandise comme nom de ligne :

```
myDF<-data.frame(dispo=c(18, 10, 5, 20),  
                 volume=c(400, 300, 200, 500),  
                 profit=c(2000, 2500, 5000, 3500),  
                 origine=c("France","Inde","France","Bresil"),  
                 row.names=c("semoule","riz","farine","lentille"))
```

Q1 : Déterminer dans ce code la façon par laquelle on associe des noms aux colonnes.

Q2 : Qu'est-ce que vous remarquez dans la zone environnement ?



Cliquer sur le cercle bleu pour afficher les détails du dataframe **myDF**.

Q3 : Comment ; appelle-t-il les lignes et colonnes des dataframes ? Vous pouvez réexécuter le même code en commentant une des colonnes.

Q4 : Quelle est le type de la colonne origine ? Correspond-il à la précédente capture ?

Considérons le code de création du dataframe **myDF**. Modifier le en ajoutant le paramètre **stringsAsFactors=TRUE**.

Q5 : Noter les modifications survenues dans **myDF** dans la zone environnement puis déduire le rôle de cet argument et sa valeur par défaut.



The screenshot shows the R Documentation page for the `data.frame` function from the `base` package. The page is titled "Data Frames" and includes a description, usage, and arguments section. The description states that `data.frame()` creates a data frame, which is a collection of variables which share many of the properties of vectors and of arrays, but which are organized in a way that allows for easy manipulation. The usage section shows the function signature: `data.frame(..., row.names = NULL, check.rows = FALSE, check.names = TRUE, fix.empty.names = TRUE, stringsAsFactors = FALSE)`. The arguments section explains that the arguments are of either the form `value` or `tag = value`. Component names are created based on the tag (if present) or the deprecated argument itself. `row.names` is a character vector of row names, or `NULL` to use the default. `check.rows` is a logical value indicating whether to check for duplicate row names. `check.names` is a logical value indicating whether to check for duplicate column names. `fix.empty.names` is a logical value indicating whether to fix empty column names. `stringsAsFactors` is a logical value indicating whether to convert character vectors to factors.

Comme pour les matrices, les noms de lignes et de colonnes peuvent être consultés ou modifiés après la création du dataframe à l'aide des fonctions **rownames()** et **colnames()**.

Q6 : Exécuter le code suivant, puis noter les modifications survenues dans **myDF**.

```
rownames(myDF)
colnames(myDF)
colnames(myDF) <- c("col1", "col2", "col3", "col4")
```

colnames(myDF)

; estimer les noms de colonnes.

Les dataframes peuvent être indexés comme des matrices. En particulier, un élément situé à l'intersection de la ligne <ligne> et la colonne <col> est accessible par <nomDataFrame>[<ligne>,<col>] où <ligne> et <col> sont soit des indices entiers ou les noms de la ligne et de la colonne respectivement.

Q7 : Afficher les valeurs de `myDF[1,2]` et `myDF["semoule","volume"]` puis noter si le résultat est le même.

```
> myDF[1,2]==myDF["semoule","volume"]
[1] TRUE
```

Q8 : Afficher les valeurs de `myDF[,4]` et `myDF[, "origine"]` puis noter si le résultat est le même.

Q9 : Vérifier si l'écriture `myDF[2,"dispo"]` est valable. Quelle est sa valeur ?

```
> myDF[2,"dispo"]
[1] 10
```

Pour accéder aux données d'un dataframe il y a d'autres moyens comme :

- Il est possible d'extraire une colonne <col> avec la syntaxe <nomDataFrame>\$<col>.

Q10 : Afficher les valeurs de `myDF$origine` puis noter le résultat.

Q11 : Afficher les valeurs de `myDF$4` et `myDF$"origine"` puis noter si le résultat.

+ Il est possible aussi d'utiliser la fonction `attach(<nomDataFrame>)` qui permet de manipuler chaque colonne par son nom comme si elle agissait d'une variable.

Q12 : Exécuter les commandes suivantes

```
> dispo <- 123 ; attach(myDF)
The following object is masked by_ .GlobalEnv:
    dispo

> dispo
[1] 123
> volume
[1] 400 300 200 500
> detach(myDF)
> volume
Error: object 'volume' not found
```

Q13 : Qu'est-ce que vous remarquez ?

attach (base)	R Documentation	Details
Attach Set of R Objects to Search Path		
Description		
The database is added to the R search path. This means that the database is searched by R when evaluating a variable, an object in the database can be accessed by simply giving their names.		
		Details
		By attaching a data frame (or list) to the search path, it is possible to refer to the variables in the data frame by their column names, rather than as components of the data frame (e.g. in the examples below, the first output is more straightforward).
		Good practice
		At least two disadvantages of altering the search path and this can easily lead to the wrong object of a particular name being found. People do often forget to detach databases.

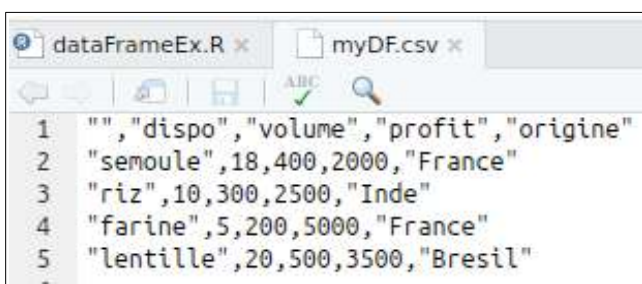
Comme règle générale, il est préférable d'éviter l'accès par `attach()` pour éviter les conflits.

B. Sauvegarder des dataframes dans des fichiers csv

Le format **CSV** ('Comma-separated values') est un format de fichier texte représentant des données tabulées sous forme de valeurs séparées par des virgules. L'utiliser sous R permet de réutiliser ses propres données pour les utiliser entre sessions ou bien de les partager avec d'autres utilisateurs utilisant R ou des tableurs. Deux fonctions peuvent être utilisées : `write.csv()` et `read.csv()`.

Q14 : Exécuter la commande `write.csv(myDF, file = "myDF.csv")` puis vérifier si le fichier csv a été créé.

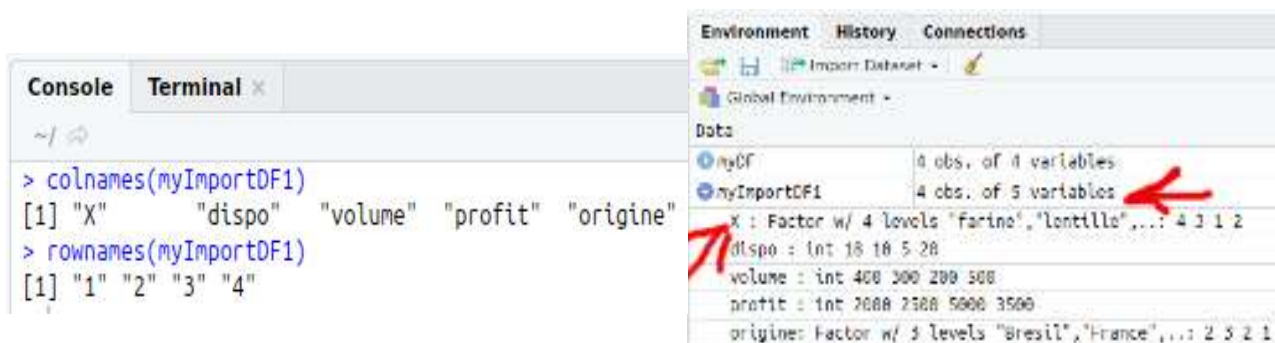
Q15 : Afficher son contenu en utilisant l'explorateur de fichier intégré à RStudio, puis en utilisant celui de votre système d'exploitation



Pour importer les données dans un dataframe identique à celui de l'export, quelques précautions doivent être prises. Les étapes suivantes vont nous permettre de le faire.

Q16 : Exécuter la commande suivante puis identifier le problème.

`myImportDF1<-read.csv(file = "myDF.csv")`



Q17 : Afficher les noms de lignes et colonnes pour voir plus clair.

`colnames(myImportDF1)`
`rownames(myImportDF1)`

Q18 : Exécuter le code suivant puis identifier la signification et utiliser pour cela le help de l'argument `row.names` dans la fonction `read.csv()`.

`myImportDF2<-read.csv(file = "myDF.csv", row.names=1)`
`colnames(myImportDF2)`
`rownames(myImportDF2)`
`str(myImportDF2$dispo)`

```

Console Terminal x
~/
> colnames(myImportDF2)
[1] "dispo" "volume" "profit" "origine"
> rownames(myImportDF2)
[1] "semoule" "riz" "farine" "lentille"
> str(myImportDF2$dispo)
int [1:4] 18 10 5 20

```

Q19 : Qu'est-ce que vous remarquez en exécutant `str(myImportDF2$dispo)`?

Exécuter maintenant le code suivant :

```

myImportDF3<-read.csv(file = "myDF.csv",row.names=1,
                      colClasses = c("character","numeric","numeric",
                                     "numeric","factor"))

colnames(myImportDF3)
rownames(myImportDF3)
str(myImportDF3$dispo)

```

Q20 : Est-ce que la variable `myImportDF3` est identique à `myDF`?

```

> myImportDF3<-read.csv(file = "myDF.csv",row.names=1,
+                       colClasses = c("character","numeric","numeric",
+                                     "numeric","factor"))
> colnames(myImportDF3)
[1] "dispo" "volume" "profit" "origine"
> rownames(myImportDF3)
[1] "semoule" "riz" "farine" "lentille"
> str(myImportDF3$dispo)
num [1:4] 18 10 5 20

```

myDF	4 obs. of 4 variables
dispo	: num 18 10 5 20
volume	: num 400 300 200 500
profit	: num 2000 2500 5000 3500
origine	: Factor w/ 3 levels "Bresil","France",...: 2 3 2 1
myImportDF1	4 obs. of 5 variables
myImportDF2	4 obs. of 4 variables
myImportDF3	4 obs. of 4 variables
dispo	: num 18 10 5 20
volume	: num 400 300 200 500
profit	: num 2000 2500 5000 3500
origine	: Factor w/ 3 levels "Bresil","France",...: 2 3 2 1

Notons que cette dernière démarche spécifiant les classes des colonnes permet aussi d'accélérer l'importation de manière significative lorsque la taille du fichier csv est importante. Donc il est recommandé de l'utiliser même si la classe par défaut supposée par l'importation est la bonne.